

AgriCoop Marketplace

Complete website purpose, user flows, features, operations, and technical reference

AgriCoop Development Team \ Technical handoff and defense-demo reference \ 2026-10-09

Table of Contents

1	Executive summary	1
2	Product purpose and problem statement	1
2.1	Purpose	1
2.2	Intended users	1
3	Development process	2
3.1	Discovery and requirements extraction	2
3.2	Engineering principles used	2
4	Current technology stack	2
5	System architecture	3
5.1	Current runtime architecture	3
5.2	Native migration architecture	4
6	Data model and persistence	4
6.1	Main entities	4
6.2	Database migrations	4
7	Authentication and authorization	4
7.1	Registration and login flow	4
7.2	Authorization layers	5
7.3	Role matrix	5
8	Public user experience	6
8.1	Landing page	6
8.2	Marketplace browsing	6
9	Seller workflow	6
9.1	Product listing lifecycle	6
9.2	Seller order and quotation activity	7
10	Buyer workflow	7
10.1	Discover and compare	7
10.2	Quotation request	7
10.3	Order flow	7
11	Payment and transaction policy	7
12	Conversations and real-time support	8
12.1	Why conversations exist	8
12.2	Access control	8
12.3	Real-time mechanism	8
13	Administration, moderation, and reporting	8
13.1	Admin dashboard	8
13.2	Product moderation queue	8
13.3	Reports	8
14	Demo data and defense preparation	8
15	Security and privacy considerations	9
15.1	Secrets	9

15.2	Data protection	9
15.3	Known security boundary	9
16	Testing and quality assurance	9
17	Deployment and operations	10
17.1	Current runtime deployment	10
17.2	Native Next.js path	10
17.3	RLS application procedure	11
18	File and directory guide	11
19	End-to-end scenario walkthroughs	12
19.1	Buyer purchases a listing	12
19.2	Buyer requests a quotation	12
19.3	Seller submits a product	12
19.4	Admin monitors the cooperative	12
20	Limitations and future roadmap	12
21	Complete route and screen map	13
22	Detailed role playbooks	14
22.1	Buyer playbook	14
22.2	Seller/Farmer playbook	14
22.3	Admin Officer / Admin playbook	15
22.4	SuperAdmin playbook	15
23	State transitions and business lifecycle	15
23.1	Product lifecycle	15
23.2	Quotation lifecycle	15
23.3	Order lifecycle	16
23.4	Transaction lifecycle	16
23.5	Conversation lifecycle	16
24	Exact business rules and validation checklist	16
25	API and procedure reference	17
26	Data dictionary and relationships	19
26.1	Identity and organizational data	19
26.2	Catalog data	19
26.3	Trading data	19
26.4	Communication and evidence data	20
27	Defense-demo walkthrough: what to show and when	20
27.1	Ten-minute public-to-Buyer demonstration	20
27.2	Ten-minute Seller and moderation demonstration	20
27.3	Five-minute Admin/reporting demonstration	20
28	Troubleshooting and operational runbook	21
28.1	Login or registration problems	21
28.2	Empty marketplace	21
28.3	Order or quotation submission fails	21
28.4	Conversation does not update	21

28.5 Build succeeds with a chunk warning 21

29 Current implementation versus final target 21

30 Document control and maintenance 22

31 Continuation brief update — 2026-10-08 23

31.1 Implemented in this update: complete logout behavior 23

31.2 Verification evidence for this update 23

31.3 Continuation brief items still pending 23

31.4 Credential and privacy boundary 24

32 Role registration and SuperAdmin provisioning update — 2026-10-09 24

32.1 Public registration roles 24

32.2 Initial SuperAdmin provisioning 25

32.3 Verification evidence 25

33 Glossary 25

34 Conclusion 26

1 Executive summary

AgriCoop is a cooperative-centered agricultural marketplace designed for local agricultural trade in Butuan City, Agusan del Norte. The platform connects Buyers and Sellers/Farmers, supports quotations and regular or bulk orders, records face-to-face payment coordination, and provides administration tools for product moderation, member oversight, reports, and support conversations.

The system was developed as a thesis-oriented prototype and evolved through several implementation stages. The current verified defense-demo runtime is a React 19 and Vite client served by an Express 4 server. tRPC provides the typed API contract; Drizzle ORM maps the business schema to MySQL/TiDB; Supabase Auth verifies bearer sessions; and an authenticated Server-Sent Events broker provides immediate conversation updates. A separate native Next.js/Tailwind/Vercel shell and a Supabase/Postgres RLS migration artifact are also included as the migration path.

Important scope statement. The current production-like runtime remains Vite + Express + Drizzle/MySQL. The native Next.js shell builds independently, but the full business-router migration to Supabase Postgres and live RLS application require a Postgres connection and further porting work.

2 Product purpose and problem statement

2.1 Purpose

AgriCoop exists to make cooperative agricultural trading easier to discover, coordinate, document, and supervise without removing the face-to-face relationships that are important to local cooperatives. It is not simply an online store. It is a coordination system for:

- making cooperative produce visible;
- comparing products, prices, units, quality grades, and available stock;
- requesting formal quotations before committing to volume purchases;
- placing regular and multi-product bulk orders;
- coordinating F2F payment and handover arrangements;
- giving Sellers a controlled listing workflow;
- giving Admins an operational view of members, products, orders, and reports; and
- preserving a written conversation trail for inquiries, quotations, and orders.

2.2 Intended users

The final application model uses four roles:

Role	Purpose and capabilities
Buyer	Browse approved listings, request quotations, place regular or bulk orders, review activity, coordinate F2F handover, and message Sellers.
Seller / Farmer	Create and update listings, maintain inventory, submit products for verification, respond to quotations, fulfill orders, and message Buyers.
Admin	Manage cooperative members, monitor operations, review and approve or reject Seller products, export reports, and oversee support threads.

Role	Purpose and capabilities
SuperAdmin	Platform-level administrative authority. SuperAdmin is not available through public registration and has precedence over normal Admin controls.

The original thesis language referred to cooperative officers and system administrators. In the current implementation, cooperative oversight is represented by Admin procedures and views, while SuperAdmin represents platform-wide administration. This consolidation avoids an unsafe public officer-registration path while preserving the required monitoring and reporting functions.

3 Development process

3.1 Discovery and requirements extraction

The project began by scanning the thesis manuscript and translating its goals into implementable product requirements. The requirements were grouped into public browsing, authentication, role workspaces, marketplace rules, quotations, ordering, payments, inventory, reporting, and deployment.

The development process then followed an incremental preservation strategy:

1. Preserve the existing landing page, marketplace browse experience, dashboards, responsive layout, and CSV reporting.
2. Add the business tables and procedures required for quotations, orders, inventory, transactions, and conversations.
3. Add custom Supabase Auth login and registration while retaining the typed server contract.
4. Add the F2F-only payment policy requested for the simplified final scope.
5. Add Seller verification and Admin moderation so unapproved products cannot appear in the public marketplace.
6. Add support conversations linked to quotation and order events.
7. Add authenticated SSE updates for connected conversation participants.
8. Add deterministic tests, seed data, environment documentation, and migration artifacts.
9. Add a native Next.js/Tailwind shell and document the remaining Postgres cutover boundary.

3.2 Engineering principles used

The implementation emphasizes typed boundaries, server-side authorization, data validation, non-destructive migrations, clear UI states, and reproducible defense-demo data. Critical operations use protected tRPC procedures rather than relying on hidden buttons. Public pages can browse only products that pass the database-level marketplace eligibility filters.

4 Current technology stack

Layer	Technology and responsibility
Client	React 19, Vite 7, Wouter routing, Tailwind CSS 4, shadcn-style Radix UI components, Lucide icons, React Query, and tRPC React bindings.

Layer	Technology and responsibility
Server	Express 4, tRPC 11, TypeScript, server-sent events for conversation updates, and storage proxy support.
Data access	Drizzle ORM with MySQL/TiDB in the current runtime. The schema is defined in <code>drizzle/schema.ts</code> .
Authentication	Supabase Auth email/password sessions. The browser forwards the active access token as a Bearer token; the server verifies it with the Supabase admin client.
Authorization	Server-side role checks in <code>server/authz.ts</code> , protected tRPC procedures, ownership checks, product moderation checks, and a target Supabase RLS artifact.
Testing	Vitest authorization, marketplace-rule, CSV-report, credential, and logout tests; TypeScript checking; Vite/esbuild production build; native Next.js build and route smoke tests.
Migration path	Next.js 16 app router shell, Tailwind v4 PostCSS integration, <code>next.config.mjs</code> , Supabase CLI configuration, and <code>supabase/migrations/20261005_rls_baseline.sql</code> .

5 System architecture

5.1 Current runtime architecture

The current runtime is a single Node process that serves the Vite client in development or built static assets in production, exposes tRPC under `/api/trpc`, and exposes the authenticated conversation stream under `/api/conversations/:conversationId/stream`.

```

Browser (React + Wouter + React Query)
  |
  | Supabase access token in Authorization header
  v
Express server
|-- /api/trpc -> typed tRPC procedures -> authz -> Drizzle/MySQL
|-- /api/conversations/:id/stream -> Supabase token check -> SSE broker
|-- storage proxy
`-- Vite dev server or static production assets
  |
  v
Supabase Auth (identity verification only in current runtime)

```

5.2 Native migration architecture

The repository also contains a separate `app/` directory for a native Next.js route tree. It currently provides a polished landing shell and working static routes for `/`, `/about`, `/marketplace`, `/support`, and `/auth`. Tailwind v4 is wired through `postcss.config.mjs`; `next:build` and `next:start` are available for Vercel-style deployment validation.

The native shell is intentionally not described as a completed business migration. Completing that migration requires porting the tRPC procedures and Drizzle/MySQL schema to Next.js route handlers or server actions backed by Supabase Postgres, then applying and testing RLS.

6 Data model and persistence

6.1 Main entities

The primary schema contains the following business concepts:

- `users`: identity-linked local profile, role, approval status, cooperative assignment, contact details, and sign-in metadata;
- `cooperatives`: cooperative name, location, description, and status;
- `products`: Seller ownership, cooperative, name, category, unit, price, stock, quality grade, description, publication status, verification status, visibility, rejection reason, and image metadata;
- `quotations`: product, Buyer, Seller, requested quantity, notes, quoted price, response note, status, and response time;
- `orders`: Buyer, Seller, cooperative, order type, status, F2F payment state, total, notes, and completion time;
- `order_items`: product-level quantities and unit prices for regular and bulk orders;
- `transactions`: order, Buyer, Seller, amount, F2F payment method, transaction status, and handover/reference notes;
- `conversations`: Buyer/Seller participants, product, order or quotation context, subject, status, and timestamps; and
- `conversation_messages`: sender, conversation, message body, and timestamps.

6.2 Database migrations

Schema changes are generated with Drizzle Kit and stored as SQL under `drizzle/`. The current migration history includes the original application tables, conversation tables, role/product moderation changes, and participant fields. Role changes are data-safe: legacy values are normalized before narrowing the role enum.

The Supabase/Postgres target artifact is separate under `supabase/rls.sql` and `supabase/migrations/20261005_rls_baseline.sql`. It assumes the Postgres profile table has an `auth_user_id` UUID that points to `auth.users(id)`.

7 Authentication and authorization

7.1 Registration and login flow

1. A visitor opens `/auth`.

2. The user chooses sign in or registration.
3. Public registration accepts only buyer or seller.
4. The server-side registration procedure creates a confirmed Supabase Auth user with `email_confirm: true`, stores role metadata, and creates the local profile.
5. The browser signs in with Supabase email/password authentication.
6. Supabase emits a session containing an access token.
7. `main.tsx` tracks the session and attaches the token to tRPC requests.
8. `server/_core/context.ts` verifies the token with Supabase, loads or creates the local profile, and exposes the role-aware user in the tRPC context.
9. Logout calls the app logout mutation and `supabase.auth.signOut()`, clears the local query cache, and invalidates protected data.

Public registration cannot create Admin or SuperAdmin accounts. Privileged roles are assigned through administrative controls. Existing server-assigned Admin/SuperAdmin roles are preserved when a Supabase session is refreshed.

7.2 Authorization layers

Authorization is intentionally repeated at several boundaries:

- client navigation and role-aware views improve usability;
- protected tRPC procedures reject unauthenticated requests;
- `requireOperationalRole` checks the allowed role set;
- ownership checks prevent users from editing another Seller's products or reading another participant's conversation;
- product purchase eligibility requires published, approved, visible products;
- self-transactions are blocked when Buyer/Seller ownership would be the same; and
- the target Postgres migration adds database-enforced RLS policies after the future cutover.

7.3 Role matrix

Area	Buyer	Seller	Admin	SuperAdmin
Marketplace browsing	Yes	Yes	Yes	Yes
Create listing	No	Yes	No	No
Submit product for review	No	Yes	No	No
Approve/reject product	No	No	Yes	Yes
Place order	Yes	Yes if eligible	No	No
Request quotation	Yes	Yes if eligible	No	No
Manage cooperative members	No	No	Yes	Yes

Area	Buyer	Seller	Admin	SuperAdmin
Platform controls	user	No	Limited	Yes
Conversation oversight	Participant	Participant	All	All

8 Public user experience

8.1 Landing page

The landing page uses a farm-fresh visual system: deep cooperative green, muted harvest green, warm off-white surfaces, rounded cards, serif display headings, and direct calls to action. Its purpose is to explain the value proposition before asking users to authenticate.

The main public routes are:

- /: product story and entry points;
- /about: How AgriCoop works and role explanation;
- /marketplace: searchable product discovery;
- /auth: Supabase sign in and registration;
- /support: inquiries and conversation entry point;
- /activity: Buyer/Seller outgoing orders and quotations; and
- /reports: report-oriented views for authorized users.

8.2 Marketplace browsing

The marketplace query filters by optional category and sort order. Public results are restricted to products whose status is published, verification status is approved, and visibility is visible. The UI supports search/filtering, category selection, price sorting, stock display, quality grade, Seller information, cart preparation, quotation requests, and F2F-only checkout coordination.

9 Seller workflow

9.1 Product listing lifecycle

1. Seller opens the dashboard and selects the listing workflow.
2. Seller enters product name, category, description, unit, price, stock quantity, quality grade, and up to ten image URLs.
3. A new record begins as a draft or pending verification according to the submission choice.
4. Seller can update the product and adjust inventory.
5. Seller submits the product for verification.
6. Admin reviews the pending queue.
7. Admin approves or rejects the listing. A rejection reason is stored for correction and resubmission.
8. Only approved and visible listings appear in the public marketplace.

This process protects marketplace quality and makes moderation explicit rather than relying on hidden UI controls.

9.2 Seller order and quotation activity

Sellers can participate as Buyers when the business rule permits it, which supports seller-to-seller and cooperative-to-cooperative trade. The system blocks self-purchase and self-quotation attempts. Seller activity includes outgoing quotations, orders, inventory changes, order fulfillment status, and support messages.

10 Buyer workflow

10.1 Discover and compare

A Buyer browses approved listings, filters by category, sorts by price, and adds eligible products to the cart. Bulk orders are normalized to a single Seller to keep fulfillment and handover coordination clear.

10.2 Quotation request

A Buyer selects a product, enters a requested quantity, and adds an optional note. The server validates product ownership, availability, and self-transaction rules before creating the quotation. A linked support conversation is opened automatically so the Buyer and Seller can clarify pricing, quantity, availability, and F2F arrangements.

The quotation lifecycle supports requested and responded states, quoted price, Seller response note, and response timestamp. Quotations can be monitored in authorized dashboards and reports.

10.3 Order flow

1. Buyer adds one or more products from the same Seller.
2. Client calculates a preview total; the server recalculates from authoritative product prices.
3. Server verifies stock, product approval/visibility, role eligibility, and self-transaction protection.
4. Order and order items are created.
5. Inventory is decremented as part of the order operation.
6. The order receives a fulfillment status and F2F payment coordination status.
7. A linked support conversation is created with a handover prompt.
8. Buyer and Seller coordinate the physical exchange and payment.
9. Seller/Admin records completion and reference notes.

11 Payment and transaction policy

The final simplified scope is deliberately F2F-only. Online payment gateway code, PayMongo intents, QR flows, and webhooks were removed from the active application path to avoid unnecessary complexity for the defense demo.

An order records:

- `paymentMethod: f2f;`
- a payment/coordination status such as pending, paid, or F2F-pending-confirmation;
- a transaction row when the payment record is documented; and
- a reference note such as “Cash handed over at cooperative office.”

This is a coordination record, not an online payment processor. The product language consistently explains that payment and handover occur face-to-face.

12 Conversations and real-time support

12.1 Why conversations exist

Conversations keep support, quotations, and handover coordination in one place. They reduce ambiguity without adding a complex chat platform. A direct inquiry creates a general thread. A quotation request creates a quotation-linked thread. An order creates an order-linked thread with a prompt to confirm availability, F2F payment, and handover details.

12.2 Access control

Buyers and Sellers can access their own participant threads. Admin and SuperAdmin can oversee threads. Server procedures validate the conversation participant IDs before returning messages or accepting replies. The restored router allows both Buyer and Seller participants to read and respond.

12.3 Real-time mechanism

The current runtime uses a lightweight in-process event broker in `server/realtime.ts`. The Express endpoint authenticates the Supabase bearer token, verifies the local user, verifies conversation membership, then keeps an SSE connection open. New messages publish an event to listeners for the same conversation. The Support page refetches message data when a message event arrives.

This is suitable for a single-process defense demo. A horizontally scaled deployment should replace the in-memory broker with Supabase Realtime, Redis pub/sub, or another hosted event system.

13 Administration, moderation, and reporting

13.1 Admin dashboard

The Admin workspace contains member validation, cooperative monitoring, inventory pulse, product moderation, reports, user role/approval management, and support oversight. SuperAdmin enters the administrative workspace with platform-level access.

13.2 Product moderation queue

Admin sees pending product submissions and can approve or reject them. Approval sets verification status to approved, visibility to visible, and publication status to published. Rejection sets verification status to rejected, visibility to hidden, archives the product, and stores an optional rejection reason.

13.3 Reports

CSV reporting covers sales, inventory, quotations, and transactions. Serialization is centralized and tested to ensure predictable headers, escaping, and row output. Reports are intended for operational review and defense-demo evidence rather than accounting replacement.

14 Demo data and defense preparation

The `scripts/seed-demo.ts` script is designed for a non-production demonstration database. It creates or reuses:

- one cooperative named for the demo environment;
- three Seller/Farmer demo profiles;

- one Buyer demo profile;
- an Admin demo profile;
- twelve products across fruits, grains, root crops, vegetables, and legumes;
- approved and visible product listings with stock and quality grades;
- requested and responded quotations;
- completed and active order examples;
- F2F transaction records with reference notes; and
- a linked support conversation for order handover.

Never run the seed command against a production database. The script is designed to be idempotent for the intended demo records, but it is still a data-writing operation and should be isolated to a defense or staging database.

Useful command:

```
pnpm seed:demo
```

15 Security and privacy considerations

15.1 Secrets

Browser-safe variables are `NEXT_PUBLIC_SUPABASE_URL` and `NEXT_PUBLIC_SUPABASE_ANON_KEY`. Server-only secrets include `SUPABASE_SERVICE_ROLE_KEY`, `DATABASE_URL`, and `JWT_SECRET`. Secret values must stay in the deployment environment and must never be committed to Git or pasted into public documentation.

15.2 Data protection

The current runtime uses server-side authorization and MySQL constraints/queries. The future Supabase target adds RLS policies for users, products, quotations, orders, transactions, conversations, and messages. The RLS functions derive the application role and cooperative from the authenticated user profile.

15.3 Known security boundary

The RLS file is a target artifact, not proof that the current MySQL runtime has Postgres RLS. It assumes a Postgres schema with `auth_user_id` profile links. Live application requires a `SUPABASE_DB_URL` or equivalent direct database connection. The current documented environment did not provide one.

16 Testing and quality assurance

The verified deterministic suite contains six Vitest files and twelve tests:

Test file	Count	Coverage
auth.logout.test.ts	1	Logout behavior and user fixture compatibility.

Test file	Count	Coverage
authz.test.ts	2	Buyer, Seller, Admin, and SuperAdmin authorization boundaries.
credentials.test.ts	1	Optional credential configuration behavior.
marketplaceRules.test.ts	5	Quotation state, order normalization, stock, transaction status, and fulfillment transitions.
reportCsv.test.ts	2	CSV serialization and escaping.
serverCredentials.test.ts	1	Network-independent server credential checks.

The standard verification commands are:

```
pnpm check
pnpm test
pnpm build
pnpm run next:build
```

The native Next shell was also started with `next start` on a test port and smoke-tested at `/`, `/about`, `/marketplace`, `/support`, and `/auth`; each returned HTTP 200.

17 Deployment and operations

17.1 Current runtime deployment

The current defense-demo application can be run locally with:

```
pnpm install
# create .env using docs/ENVIRONMENT.md
pnpm dev
```

For a production-like build:

```
pnpm build
pnpm start
```

The runtime expects a MySQL/TiDB `DATABASE_URL`, Supabase public Auth values, and a server Supabase service role key. It is not yet a standalone Next.js + Supabase Postgres deployment.

17.2 Native Next.js path

The repository includes:

- `app/layout.tsx` and `app/page.tsx`;
- native routes under `app/about`, `app/marketplace`, `app/support`, and `app/auth`;
- `next.config.mjs`;
- `postcss.config.mjs` with Tailwind v4; and
- `next:dev`, `next:build`, and `next:start` scripts.

These files establish a deployable Next.js shell. The remaining migration work is to connect these routes to Supabase Auth, port business procedures to Postgres-backed server actions or route handlers, replace the MySQL schema, and move realtime support to a hosted service.

17.3 RLS application procedure

After provisioning the target Supabase project and obtaining a direct Postgres connection privately:

```
export SUPABASE_DB_URL='postgresql://...'
supabase db push --db-url "$SUPABASE_DB_URL"
```

Then verify with authenticated Buyer, Seller, Admin, and SuperAdmin sessions. Confirm that an unapproved product is not visible, a Buyer cannot read another Buyer's order, a Seller cannot edit another Seller's product, and Admin/SuperAdmin oversight works as intended.

18 File and directory guide

Path	Responsibility
<code>client/src/pages/</code>	Public pages, marketplace, dashboards, activity, reports, support, and auth.
<code>client/src/components/AgriShell.tsx</code>	Public/workspace shell, navigation, and shared page framing.
<code>client/src/main.tsx</code>	React bootstrap, Supabase session forwarding, React Query, and tRPC client.
<code>server/routers.ts</code>	Typed procedures for auth, marketplace, products, quotations, orders, conversations, reports, and admin controls.
<code>server/_core/context.ts</code>	Supabase bearer verification and local role-aware context construction.
<code>server/db.ts</code>	Drizzle query helpers, marketplace filters, conversations, reports, and upserts.
<code>server/authz.ts</code>	Role and ownership authorization helpers.
<code>server/realtime.ts</code>	In-process conversation event broker for SSE.
<code>drizzle/schema.ts</code>	Current MySQL/TiDB schema definitions.
<code>drizzle/*.sql</code>	Generated and reviewed database migrations.

Path	Responsibility
scripts/seed-demo.ts	Defense-demo cooperative, accounts, products, quotations, orders, transactions, and conversation data.
supabase/rls.sql	Target Postgres RLS policy artifact.
supabase/migrations/	Supabase CLI migration wrapper for the target Postgres database.
app/	Native Next.js deployment shell and routes.
docs/ENVIRONMENT.md	Environment variable and migration guidance.

19 End-to-end scenario walkthroughs

19.1 Buyer purchases a listing

Buyer signs in, opens the marketplace, filters to a product category, adds a product, confirms quantity, and submits an F2F order. The server recalculates the total, verifies stock and eligibility, creates order items, decrements inventory, records F2F coordination, and opens a support thread. Buyer and Seller then use the thread to confirm pickup and handover. Seller or Admin later records the transaction note and completion status.

19.2 Buyer requests a quotation

Buyer chooses a product, enters a requested quantity and note, and submits. The server verifies the product and ownership rules, creates a quotation in the requested state, and opens a quotation-linked conversation. Seller reviews the request, responds with a price and note, and the Buyer sees the response in activity and support views.

19.3 Seller submits a product

Seller saves product details and submits the listing for verification. Admin sees the pending listing, checks product information, and approves or rejects it. A rejected listing remains hidden and retains the rejection reason. After correction and resubmission, approval makes it eligible for public marketplace discovery.

19.4 Admin monitors the cooperative

Admin opens the dashboard to review member status, inventory pulse, quotations, orders, transactions, pending product submissions, and report snapshots. Admin can export CSV reports and oversee support conversations. SuperAdmin can perform platform-wide role and cooperative controls.

20 Limitations and future roadmap

The following items are intentionally identified rather than hidden:

1. The current business runtime is MySQL/TiDB through Drizzle, not Supabase Postgres.
2. The RLS file is ready but has not been applied to a live Postgres target because no connection URL was provided.
3. The native Next.js app is a deployment shell; the full marketplace business procedures still run in the existing Express/tRPC runtime.

4. The in-process SSE broker is appropriate for one process but should be replaced for horizontal scaling.
5. Product images are represented by URL metadata and require a production storage policy and upload UX.
6. F2F payment records document coordination and handover; they do not process funds online.
7. Demo seed data should remain isolated from production data.

Recommended roadmap:

- Port `drizzle/schema.ts` to Supabase Postgres and add stable `auth_user_id` profile links.
- Convert tRPC procedures into Next.js route handlers/server actions or retain a separately deployed typed API.
- Apply and integration-test RLS with real authenticated sessions.
- Replace the in-process event broker with Supabase Realtime.
- Add production storage for product images and audit logging for Admin actions.
- Add Playwright end-to-end tests for Buyer, Seller, Admin, and SuperAdmin walkthroughs.

21 Complete route and screen map

This section explains what a visitor or signed-in user can do on every application route. The current verified runtime uses Wouter routes in `client/src/App.tsx`; the native Next.js shell contains a parallel public route structure but is not yet the connected business runtime.

Route	Who uses it	Purpose and expected actions
/	Everyone	Explains the cooperative value proposition and links to marketplace discovery, How It Works, support, and sign-in.
/about	Everyone	Explains the marketplace model, roles, F2F coordination, and how cooperative participation works.
/marketplace	Everyone; signed-in users get actions	Search and filter approved, published, visible listings; view price, unit, stock, quality, and Seller context; request a quotation or prepare an order.
/auth	Visitors	Sign in or register with email and password through the custom Supabase Auth flow.
/activity	Buyer and Seller	Review outgoing quotations, orders, status, payment coordination, and linked activity.
/support	Authenticated participants; Admin/SuperAdmin oversight	Create inquiries, open linked threads, read messages, send replies, and coordinate quotation/order details.
/dashboard	Authenticated users; role-aware workspace	Seller listing/inventory controls, Buyer/Seller activity, cooperative oversight, moderation, user controls, and role-specific operations.

Route	Who uses it	Purpose and expected actions
/reports	Admin/SuperAdmin	View and export CSV reports for sales, inventory, quotations, and transactions.
/404	Everyone	Friendly not-found state with a route back to the application.

The navigation is intentionally different for public and workspace contexts. Public pages use a lightweight top navigation and clear calls to action. Authenticated workspaces use role-aware navigation so users can reach marketplace, activity, support, dashboard, and reports without losing context.

22 Detailed role playbooks

22.1 Buyer playbook

The Buyer is the demand-side participant. A Buyer may be a household, institutional purchaser, cooperative, or an approved Seller acting as a purchaser from another Seller. The platform does not assume that every Buyer is anonymous: the account makes quotations, orders, and support history attributable.

1. Open the landing page and choose **Explore fresh listings**.
2. Search by product name or narrow the catalog by category.
3. Compare unit, price, stock, quality grade, and Seller information.
4. Choose either a quotation request or an order. A quotation is appropriate when price or availability needs discussion; an order is appropriate when the terms are clear.
5. Enter a valid requested quantity and optional note. The server validates the quantity, product eligibility, ownership rules, and total.
6. Submit. The system creates the quotation or order and opens a linked support conversation where applicable.
7. Use Activity to follow status and Support to coordinate pickup, handover, timing, and payment reference notes.
8. After physical handover, confirm or ask the Seller/Admin to record the F2F transaction completion according to the operational procedure.

A Buyer must not be able to buy their own listing or request a quotation from themselves. The server checks ownership; hiding a button is not the security control.

22.2 Seller/Farmer playbook

The Seller is both a producer and, when allowed, a marketplace participant who may purchase from another Seller. This supports farmer-to-farmer and cooperative-to-cooperative trading while blocking self-transactions.

1. Open Dashboard and choose the product-management workflow.
2. Create a product with name, category, description, unit, price, stock, quality grade, and image references.
3. Save as draft when information is incomplete, or submit for review when the listing is ready.
4. Monitor the verification state. A pending listing is not public; an approved and visible listing is public; a rejected listing remains hidden and includes the reason for correction.

5. Update stock and product details through the Seller-owned controls.
6. Review incoming quotations, provide a quoted price and response note, and transition the quotation to responded, accepted, or declined as supported by the active workflow.
7. Review incoming orders, coordinate readiness and F2F handover, and use the status controls available to the Seller role.
8. Reply in linked support conversations so the Buyer has a single written coordination trail.

22.3 Admin Officer / Admin playbook

The current runtime labels the cooperative oversight role admin; the target manuscript name is Admin Officer. The responsibility is cooperative-level oversight: membership validation, product moderation, operational monitoring, support oversight, and report export.

The Admin reviews the moderation queue rather than directly editing Seller content invisibly. Approval changes a product to approved/visible/published. Rejection changes it to rejected/hidden/archived and preserves a correction reason. This makes the decision explainable to the Seller.

The Admin workspace is also the operational safety net for conversations and F2F coordination. Admin can inspect threads, monitor order and quotation states, review the inventory pulse, update member approval status where authorized, and export evidence for the cooperative.

22.4 SuperAdmin playbook

SuperAdmin is the platform-level authority. It is not a public registration option. In the current runtime it can access the broader administrative procedures and platform oversight paths; in the target model it will approve Admin Officers, manage platform-level configuration, and retain visibility across cooperatives. SuperAdmin actions must remain auditable in the target system. They should not be implemented as a public role-selection shortcut because that would allow privilege escalation.

23 State transitions and business lifecycle

23.1 Product lifecycle

```
DRAFT -> PENDING REVIEW -> APPROVED/PUBLISHED/VISIBLE
      \-> REJECTED/ARCHIVED/HIDDEN -> DRAFT/PENDING REVIEW
APPROVED/PUBLISHED/VISIBLE -> ARCHIVED/HIDDEN (Seller/Admin action)
```

A product is publicly discoverable only when all three public eligibility conditions are true: publication status is published, verification status is approved, and visibility is visible. A product can exist in the database without being public.

23.2 Quotation lifecycle

```
REQUESTED -> RESPONDED -> ACCEPTED
      \-> DECLINED
```

The request contains the product, Buyer, Seller, quantity, and optional Buyer note. The response may include quoted price, Seller note, and response timestamp. The linked conversation gives the participants a place to clarify terms without losing the quotation record.

23.3 Order lifecycle

```

SUBMITTED -> CONFIRMED -> READY -> COMPLETED
      |           |           |
      +-----+-----+----> CANCELLED
  
```

The current prototype creates order items and applies its existing placement-time inventory behavior. The target specification requires a different invariant: stock must be deducted only at completion, after a fresh stock check and with final quantities. This distinction is important and is intentionally recorded as a migration gap, not hidden as if already solved.

23.4 Transaction lifecycle

F2F payment is a coordination record rather than a gateway payment. The order carries the coordination status and the transaction carries a reference note. Typical operational progression is payment coordinated or F2F-pending-confirmation, followed by recorded/paid after physical handover is verified. No online intent, QR checkout, or webhook is active in the current scope.

23.5 Conversation lifecycle

```

OPEN -> CLOSED
  
```

A conversation may be created manually as an inquiry or automatically from a quotation or order event. It stores a subject and context identifiers so participants can understand what the thread is about. The target architecture requires three explicit members: Buyer, Seller, and the Admin Officer of the Seller's cooperative. The current runtime has Buyer/Seller participation plus Admin/SuperAdmin oversight, so this is a known migration gap.

24 Exact business rules and validation checklist

The following rules explain what the system should accept, reject, and why.

Rule	Expected behavior	Current evidence
Public product visibility	Only published + approved + visible products are returned to public marketplace queries.	Implemented and covered by active filters.
Product ownership	Seller may manage own products; cannot edit another Seller's product.	Server ownership checks; target RLS artifact exists.

Rule	Expected behavior	Current evidence
Self-purchase	Buyer/Seller pair with the same account is rejected.	Implemented in marketplace rules; target database proof pending.
Self-quotation	A Seller cannot request a quotation from their own product.	Implemented in marketplace rules; target database proof pending.
Order total	Server recalculates from authoritative prices; client preview is not trusted.	Implemented in current router path.
F2F payment	Payment method is fixed to F2F/external coordination; store reference notes.	Implemented; gateway path removed.
Quantity limits	Target product fields min/max quantity must be enforced for regular and bulk orders.	Not yet in current MySQL schema.
Completion stock	Target stock deduction occurs exactly once at completion after fresh check.	Not yet proven; current placement-time deduction differs.
Bulk order scope	All items in one bulk order belong to exactly one Seller.	Business intent exists; database invariant pending.
Role creation	Public registration is Buyer or Seller only; privileged roles are assigned administratively.	Partially implemented; approval/email target flow pending.

Validation is also a user-experience concern. Every important form should communicate required fields, invalid values, loading/disabled submission state, server failure, and success. The current application uses typed Zod inputs in protected procedures and toast/error feedback in the client; the target Next.js implementation must preserve these states in every server action or route handler.

25 API and procedure reference

The current backend is a tRPC router. Procedure names below are the implemented contract surface observed in `server/routers.ts`.

Router / procedure	Access	Purpose
<code>auth.me</code>	Public	Return the current local user context when available.
<code>auth.register</code>	Public	Validate registration fields, create the Supabase Auth user/local profile path, and persist role metadata according to the active implementation.

Router / procedure	Access	Purpose
auth.logout	Public	End the local session flow.
conversations.mine	Protected	List threads available to the signed-in user; Admin/SuperAdmin may oversee broader threads.
conversations.messages	Protected	Read a conversation after membership/oversight checks.
conversations.create	Protected	Open a direct inquiry thread.
conversations.send	Protected	Validate and send a message to an allowed thread.
account.myProfile	Protected	Read the signed-in profile.
account.updateProfile	Protected	Update allowed profile fields.
cooperatives.list	Public	List cooperative records used by public and onboarding views.
cooperatives.create	Protected/admin	Create a cooperative through an authorized procedure.
marketplace.list	Public	Filter and sort eligible marketplace products.
marketplace.categories	Public	Return available product categories.
products.mine	Protected Seller	List the Seller's own products and statuses.
products.create	Protected Seller	Create a draft or pending product.
products.submit	Protected Seller	Submit a product for moderation.
products.update	Protected owner	Edit owned product fields.
products.archive	Protected owner/ admin	Hide/archive a product.
quotations.buyerMine	Protected	List quotations requested by the current user.
quotations.sellerMine	Protected	List quotations directed to the current Seller.
quotations.request	Protected buyer- eligible	Create a quotation and its coordination path.
quotations.respond	Protected Seller	Respond with quoted terms and note.
orders.buyerMine	Protected	List orders placed by the current Buyer.
orders.sellerMine	Protected	List orders directed to the current Seller.
orders.create	Protected buyer- eligible	Validate and create an order from item inputs.
orders.updateStatus	Protected Seller/ admin	Advance or cancel order according to role and transition checks.
cooperativeAdmin.overview	Protected Admin/ SuperAdmin	Return cooperative monitoring data.

Router / procedure	Access	Purpose
cooperativeAdmin.updateMemberStatus	Protected Admin/ SuperAdmin	Update member approval/operational state.
admin.overview	Protected Admin/ SuperAdmin	Return administrative overview data.
admin.reviewProduct	Protected Admin/ SuperAdmin	Approve or reject a product and preserve rejection reason.
admin.updateUser	Protected Admin/ SuperAdmin	Apply authorized user/role/cooperative administration.

In the target Next.js version, each procedure should become a server action or route handler with the same contract concepts: validated input, authenticated identity, role/cooperative authorization, deterministic output, and consistent error semantics.

26 Data dictionary and relationships

26.1 Identity and organizational data

- **cooperatives:** the organizational boundary. Stores name, description, location, active/inactive status, and creation time.
- **users** in the current runtime: local profile linked to an external identity through openId; stores name, email, login method, role, approval status, cooperative, contact, address, and sign-in timestamps.
- **profiles** in the target model: replaces/ports the local profile and must include `auth_user_id` uuid references `auth.users(id)` plus the exact target role enum.

26.2 Catalog data

- **products:** Seller ownership, cooperative, name, category, description, unit, integer price in cents, stock quantity, quality grade, publication status, verification status, visibility, rejection reason, primary image reference, and timestamps.
- **quality grade:** the current prototype uses values such as premium/standard; the target brief requires `grade_a` and `grade_b` so produce that fails strict standards can still be truthfully listed and priced.
- **image paths:** the target architecture should store Supabase Storage paths, not file bytes in Postgres. Uploads need type, size, ownership, and private/public bucket policy checks.

26.3 Trading data

- **quotations:** product, Buyer, Seller, requested quantity, Buyer note, quoted price, Seller note, status, and response timestamp.
- **orders:** Buyer, Seller, cooperative, regular/bulk type, status, payment coordination fields, total, Buyer note, completion timestamp, and timestamps.
- **order_items:** order, product, requested quantity, final quantity nullable until completion in the target schema, and unit price snapshot.
- **transactions:** one order-level payment coordination record, Buyer, Seller, amount, F2F reference note, status, method, and completion time.

26.4 Communication and evidence data

- **conversations:** subject plus Buyer/Seller/product/order/quotation context in the current schema. The target schema adds explicit participant membership so the three authorized people are queryable.
- **conversation_participants:** target join table for explicit membership and RLS checks.
- **conversation_messages:** conversation, sender, body, and creation time. Messages must be validated for length, membership, and rate limits.
- **audit_log:** target evidence table for completion, cancellation, approvals, and role changes.
- **analytics_events:** target first-party event table for listing viewed, quotation requested, order placed, message sent, and order completed, scoped to cooperative. It replaces third-party tracking.

27 Defense-demo walkthrough: what to show and when

27.1 Ten-minute public-to-Buyer demonstration

1. Start at the landing page and explain the cooperative problem: discoverability, trust, and coordination.
2. Open Marketplace, filter by category, and point out that only approved/visible listings are public.
3. Open Auth, sign in as the Buyer demo account, then return to the marketplace.
4. Request a quotation for a volume product and show the requested state in Activity.
5. Open Support and show that the request has a context-linked conversation.
6. Create a regular or same-Seller bulk order with F2F coordination.
7. Show the order status and payment reference note path rather than presenting an online gateway.

27.2 Ten-minute Seller and moderation demonstration

1. Sign in as a Seller and open Dashboard.
2. Create or edit a product, deliberately show stock, unit, price, grade, and submission choice.
3. Submit it so its moderation state is visible.
4. Switch to Admin and open the moderation queue.
5. Approve one listing and reject another with a reason.
6. Return to Marketplace and show that the approved listing is visible while the rejected one is not.
7. Open a quotation as Seller, respond with price/note, and show the Buyer-facing activity change.

27.3 Five-minute Admin/reporting demonstration

1. Open the Admin workspace and identify cooperative overview, members, products, orders, quotations, transactions, and conversations.
2. Use the report controls to export sales, inventory, quotation, and transaction CSVs.
3. Explain that reports are operational evidence, not a replacement for accounting or a payment processor.
4. Show the known target-stack boundary honestly if asked: the current runtime remains Vite/Express/MySQL while Next.js/Supabase artifacts are being migrated.

28 Troubleshooting and operational runbook

28.1 Login or registration problems

Check that the browser-safe Supabase URL and anon key are present. Confirm that the server-only service-role key is available only to the server. If Supabase rejects registration because of provider rate limits, do not disable security by committing keys or bypassing verification; configure the provider/SMTP settings in the Supabase project and use the documented environment variables.

The current simplified demo path has a known email-confirmation divergence documented in the review package. The target behavior is email verification ON and public sign-up restricted to Buyer/Seller.

28.2 Empty marketplace

An empty marketplace can be correct when products are draft, pending, rejected, hidden, archived, or out of the selected filter. Check product verification status, visibility, publication status, cooperative assignment, and the active query filters before assuming the database is empty.

28.3 Order or quotation submission fails

Check authentication first, then role eligibility, product approval/visibility, self-purchase/self-quotation prevention, quantity/stock, and database availability. The server—not the client cart total—must be treated as authoritative.

28.4 Conversation does not update

The current runtime uses an authenticated SSE connection backed by an in-process broker. Verify the user is a participant, the stream endpoint can validate the Supabase bearer token, and the server process has not been restarted. In a horizontally scaled deployment, replace the in-memory broker with Supabase Realtime or another shared pub/sub service.

28.5 Build succeeds with a chunk warning

The verified Vite build passes but reports a main JavaScript chunk of approximately 752 kB uncompressed. The target quality plan includes route-level code splitting and manual chunks, then a Lighthouse/slow-3G verification rather than simply hiding the warning.

29 Current implementation versus final target

This manual distinguishes capability from target conformance so a defense presentation does not overclaim.

Area	Verified current state	Target work remaining
Visual experience	Approved farm-fresh responsive public and workspace UI exists.	Preserve visual design while migrating data/actions.
Business runtime	React/Vite + Express/tRPC + Drizzle/MySQL.	Native Next.js App Router + Supabase Postgres actions/handlers.

Area	Verified current state	Target work remaining
Authentication	Custom Supabase Auth bearer-token path exists; simplified registration behavior remains documented.	Email verification ON, exact role onboarding, approval flows, and auth rate limiting.
Authorization	Server-side role/ownership checks and target RLS artifact.	Apply and prove RLS on every target table with authenticated integration tests.
Payments	F2F-only coordination; no active gateway/webhook.	Keep F2F-only and complete final transaction/completion invariants.
Inventory	Current placement-time deduction behavior.	Deduct only once at completion with fresh stock and concurrency protection.
Chat	Buyer/Seller participant conversations with Admin/SuperAdmin oversight and SSE.	Explicit three-party membership and Supabase Realtime.
Storage	URL/image metadata and existing storage-related runtime artifacts.	Private Supabase Storage bucket, safe uploads, image paths, and policies.
Testing	TypeScript, 6 Vitest files/12 tests, build; review package records results.	Lint, RLS tests, Playwright role flows, Lighthouse, slow-3G, clean-clone proof.

30 Document control and maintenance

This PDF is a point-in-time operational and technical manual. Update it when any of the following changes: role names or permissions, payment policy, order/inventory lifecycle, database schema, authentication behavior, route map, deployment commands, or known-gap status. Each update should record the date, verified commands, changed files, and whether the update describes the current runtime or the target migration.

The authoritative source files for a future maintainer are the application code, `README.md`, `docs/ENVIRONMENT.md`, the review package, `drizzle/schema.ts`, `server/routers.ts`, `server/authz.ts`, `server/marketplaceRules.ts`, `scripts/seed-demo.ts`, and `supabase/rls.sql`. This PDF explains those sources; it does not replace them.

31 Continuation brief update — 2026-10-08

The continuation brief requested a broad refinement of authentication, four-role permissions, account verification, profile management, moderation, messaging, security, seed data, and testing. This section records the verified work completed in this update and separates it from work that remains pending. It is included so the PDF does not imply that a requirement is complete merely because it has been specified.

31.1 Implemented in this update: complete logout behavior

The logout path was hardened in both the browser and server layers:

- the client calls the server logout mutation and clears the application session cookie;
- Supabase Auth is signed out with global scope when configured, invalidating the refresh-session path rather than only hiding the current user in React state;
- Supabase auth entries are removed from localStorage and sessionStorage;
- the tRPC user query is cleared and the remaining query cache is invalidated;
- the server response sends `Cache-Control: no-store` so protected responses are not intentionally reused after logout;
- the browser redirects with `window.location.replace("/auth")`, which avoids adding the protected page to the forward history path;
- the protected-query error boundary still redirects an unauthenticated request to `/auth`.

The fix is designed to satisfy the required scenario: **login** → **dashboard** → **logout** → **login page** → **refresh** → **still logged out**. A browser-level automated test against a live Supabase session is still required for final proof; the available deterministic test verifies the server cookie-clearing and no-store behavior.

31.2 Verification evidence for this update

Check	Result	Evidence
TypeScript	PASS	<code>pnpm check</code> exit code 0.
Focused logout test	PASS	<code>server/auth/logout.test.ts</code> : clears session cookie and asserts <code>Cache-Control: no-store</code> .
Full deterministic suite	PASS	6 test files; 12 tests passed; 0 failed; 0 skipped.
Production build	PASS with warning	<code>pnpm build</code> exit code 0; Vite reports a 752.51 kB main chunk.
Live Supabase logout	NOT DONE	Requires an authenticated browser session and a live Supabase environment.

31.3 Continuation brief items still pending

The remaining requested items require a larger migration slice and, for sensitive verification functions, provider credentials and a live target. They are not claimed as complete in this manual:

- Exact buyer, seller, `admin_officer`, and superadmin role persistence with Admin Officer approval by SuperAdmin.
- Email verification with expiring single-use tokens and resend rate limits. The current simplified demo path still has the documented email-confirmation divergence.

- Phone/SMS OTP delivery, cooldown, attempt limits, and abuse protection. This requires an authorized SMS provider or a test double explicitly configured for local tests.
- Government ID upload and review. This requires private Supabase Storage policies, file validation, consent messaging, authorized access logging, and a live Postgres/Storage target. Government ID images and numbers must never be placed in this PDF or public API responses.
- Buyer verification gating before purchase and Seller verification gating before listing submission/publication.
- A separate Profile and Change Password workspace for every role, with provider-supported password/session invalidation.
- Explicit three-member conversation records, unread/read state, admin moderation filters, and Supabase Realtime. The current runtime provides Buyer/Seller conversations with Admin/SuperAdmin oversight through authenticated SSE.
- Audit log and first-party analytics event persistence for security actions, moderation, verification, and marketplace conversions.
- The ten named sample listings can be represented in the isolated demo seed, but the current seed remains the existing defense dataset; it must not be run against production.
- Live RLS integration tests, Playwright flows for all four roles, Lighthouse evidence, slow-3G evidence, and clean-clone verification.

31.4 Credential and privacy boundary

No plaintext passwords, service-role keys, SMS secrets, government IDs, or authentication tokens are stored in the application source or this PDF. Initial privileged accounts must be created through a secure, server-only seed configuration with temporary credentials and forced password change; the credentials themselves are intentionally excluded from documentation. The existing project credentials supplied through chat are not repeated here.

32 Role registration and SuperAdmin provisioning update — 2026-10-09

32.1 Public registration roles

The public registration form now offers three safe choices:

- **Buyer / Consumer** — approved for ordinary marketplace participation after registration.
- **Seller / Farmer** — registered pending approval and subject to the existing seller controls.
- **Cooperative Admin Officer** — publicly registerable, but always created with `approvalStatus = pending`.

SuperAdmin is not a public registration option. This prevents an unauthenticated visitor from selecting the highest-privilege role.

A pending Cooperative Admin Officer may authenticate, but server-side operational authorization rejects admin procedures until a SuperAdmin changes the record to approved. This rule is enforced in `requireOperationalRole`, not only in navigation or visual route guards. Existing approved Admin Officer records continue to work. A profile update cannot be used by a Buyer or Seller to self-promote into Admin Officer; the public registration role and profile-edit role schemas are intentionally separate.

32.2 Initial SuperAdmin provisioning

The project now includes `scripts/seed-superadmins.ts` and the `pnpm seed:superadmins` command. The script provisions the two owner-specified SuperAdmin identities through Supabase Auth and synchronizes approved superadmin rows into the application database.

Passwords are **not hard-coded**, committed, printed, returned by an API, or included in this manual. The script reads them only from these server-only environment variables:

- `SUPERADMIN_JOAN_PASSWORD`
- `SUPERADMIN_EROL_PASSWORD`

The command also requires `NEXT_PUBLIC_SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`, and the application `DATABASE_URL`. It creates or updates the matching Auth account, sets the role metadata to superadmin, confirms the email for the initial seed, and writes only non-secret provisioning status to the terminal. Run it only from a trusted server environment, then rotate the temporary credentials and enable the organization's normal password policy.

32.3 Verification evidence

Check	Result	Evidence
TypeScript	PASS	<code>pnpm check</code> exit code 0.
Authorization regression	PASS	<code>server/authz.test.ts</code> : 3 tests pass, including pending Admin Officer rejection.
Full deterministic suite	PASS	6 test files; 13 tests passed; 0 failed; 0 skipped.
Production build	PASS with warning	<code>pnpm build</code> exit code 0; existing large-chunk warning remains.
Live SuperAdmin provisioning	NOT RUN	Requires the server-only Supabase and database environment; no password was written to the repository or command line.

33 Glossary

Term	Meaning in AgriCoop
F2F	Face-to-face payment and handover coordination.
RLS	Row Level Security, a database policy mechanism planned for Supabase Postgres.
tRPC	Typed client/server API contract used by the current Express runtime.
Drizzle	TypeScript ORM used for current MySQL/TiDB persistence.
SSE	Server-Sent Events, the one-way event stream used for immediate support updates.
Verification	Admin review state for Seller product submissions.
Visibility	Whether an approved product is allowed to appear in the marketplace.
Quotation	A request for price and availability before an order commitment.

Term	Meaning in AgriCoop
Bulk order	An order containing multiple products from one Seller with a combined total.

34 Conclusion

AgriCoop is a complete thesis-ready cooperative marketplace prototype with public browsing, custom Supabase Auth, four role-aware workspaces, Seller moderation, quotation and order workflows, inventory rules, F2F transaction records, CSV reporting, support conversations, realtime updates, seed data, tests, and a documented migration path toward native Next.js and Supabase Postgres.

Its most important design decision is to preserve cooperative coordination rather than treat the platform as a generic checkout system. Product approval protects trust, server-side authorization protects ownership, support threads preserve context, and F2F records keep the payment model aligned with the simplified defense-demo requirement.